LangGraph: Graph-Based Orchestration for Stateful LLM Applications

and its Applications

CAQ413 Seminar

Seminar Report submitted in partial fulfillment for the degree
B.Tech in Computer Science and Engineering (Artificial Intelligence)

Submitted by

Nikhil Jimmy Thomas (Reg No: MUT23CA055)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
(Affiliated to APJ Abdul Kalam Technological University)

October 2026

Muthoot Institute of Technology and Science (Autonomous)
Kochi

Varikoli P.O., Puthencruz-682308

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

CERTIFICATE

This is to certify that the seminar report entitled “LangGraph: Graph-Based Orchestration for Stateful LLM Applications and its Applications” submitted by Nikhil Jimmy Thomas (Reg No: MUT23CA055) of Semester VII B.Tech Computer Science and Engineering (AI) is a bonafide record of the work done under my supervision during the academic year 2026–2027.

Guide	**Coordinators**	**Head of the Department**
Ms. Nimmi M K	Ms. Thasni K N & Ms. Vishnupriya Vijay	Dr. Cerene Mariam Abraham
Asst. Professor	Asst. Professors	Associate Professor
Dept. of AI & DS	Dept. of AI & DS	Dept. of AI & DS

ACKNOWLEDGEMENTS

This seminar report is the outcome of my own effort, but it would not have taken shape without the support and guidance of several people and institutions. It is now my privilege to acknowledge their contributions.

First and foremost, I would like to express my heartfelt gratitude to my seminar guide, Asst. Prof. *Ms. Nimmi M K*, for the continuous encouragement, guidance and patience shown throughout this seminar. The suggestions I received at each stage helped me move from a broad interest in AI agents to a focused study of LangGraph. I extend my sincere thanks to my seminar coordinators, Asst. Prof. *Ms. Thasni K N* and Asst. Prof. *Ms. Vishnupriya Vijay*, for reviewing my work critically and offering feedback that improved both the presentation and this report.

I am also grateful to Dr. Cerene Mariam Abraham, Head of the Department of Artificial Intelligence and Data Science, for her constant support and for fostering an environment that encourages learning. I would like to express my sincere gratitude to Dr. Neelakantan P. C., Principal, for his guidance and for the academic framework that made this seminar possible.

I am thankful to the management of Muthoot Institute of Technology & Science for the facilities and resources provided, and to all the faculty and staff of the Artificial Intelligence and Data Science Department for a supportive learning environment.

I also remain indebted to my friends and classmates of S7 CSAI, whose discussions, questions and company made this work more enjoyable. Finally, I thank everyone who, knowingly or unknowingly, contributed to the completion of this seminar report.

Nikhil Jimmy Thomas
(Reg. No: MUT23CA055)

Abstract

Large language models (LLMs) can now reason, write text and call external tools, but most real tasks need more than a single model call. They need several steps, decisions about what to do next, retries when something goes wrong, and memory of what has already happened. Simple prompt chains run in a fixed order and cannot loop, while free-running agents are flexible but hard to control and debug. LangGraph, an open-source library from LangChain Inc., takes a middle path: the developer draws the application as a graph of nodes (steps) and edges (transitions) that share one state object, and the LLM does its work inside each node. This seminar studies LangGraph through the base paper by Duan and Wang, "Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI" (arXiv:2411.18241, 2024). The paper combines LangGraph, which controls the flow of a multi-agent application, with CrewAI, which organises role-based agents into a team. It illustrates the combination with three cases: an email assistant that checks, drafts and sends replies, a code generation and code review loop, and an automatic work-order (ticket) auditing system. The report explains LangGraph's core concepts, reviews six related papers on LLM agents, describes the architecture and workflow proposed in the base paper, and discusses its findings, advantages, limitations and applications. A key observation is that the paper's results are qualitative case studies rather than measured benchmarks, which leaves room for future evaluation.

Keywords: LangGraph, CrewAI, Large Language Models, Multi-Agent Systems, Agent Orchestration, Generative AI.

Contents

List of Figures vi

List of Tables vii

List of Abbreviations viii

1 Introduction 1

2 Literature Survey 3
- 2.1 ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2023) . . . 3
- 2.2 Reflexion: Language Agents with Verbal Reinforcement Learning (Shinn et al., 2023) . . . 3
- 2.3 AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation (Wu et al., 2023) . . . 4
- 2.4 MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework (Hong et al., 2024) . . . 4
- 2.5 A Survey on Large Language Model based Autonomous Agents (Wang et al., 2023) . . . 4
- 2.6 Agent AI with LangGraph: A Modular Framework for Enhancing Machine Translation Using LLMs (Wang and Duan, 2024) . . . 4

3 Materials and Methods 6
- 3.1 LangGraph: Core Concepts . . . 6
- 3.2 CrewAI Framework . . . 7
- 3.3 Development Environment . . . 8
- 3.4 Evaluation Criteria . . . 8

4 Methodology 9
- 4.1 Design Rationale . . . 9
- 4.2 System Architecture . . . 9
- 4.3 Integrating LangGraph with CrewAI . . . 10
- 4.4 Workflow . . . 10
- 4.5 Worked Example: Automatic Email Handling . . . 11

5 Results and Performance 15
- 5.1 Application Case 1: Code Generation with Code Review . . . 15
- 5.2 Application Case 2: Automatic Work-Order Processing . . . 16
- 5.3 Key Findings . . . 16

5.4 Comparison with Other Approaches 17
5.5 Discussion 17

6 Advantages **18**

7 Disadvantages **19**

8 Applications **20**

9 Conclusion **21**

A Bibliography **22**

B Presentation Slides **24**

C Sample Reference Papers **27**

D Viva Questions and Answers **28**

E Similarity Index Report **30**

F Alignment with Sustainable Development Goals (SDG) **31**

List of Figures

3.1 A minimal LangGraph agent: nodes, a conditional edge (router) and a loop . 7

4.1 Two-layer architecture: LangGraph controls the flow, CrewAI performs the work inside each node 10
4.2 Workflow of the LangGraph+CrewAI system 11
4.3 Email workflow graph built with LangGraph+CrewAI (source: Duan and Wang [1], Fig. 1) 12
4.4 LangGraph+CrewAI workflow code (source: [1], Fig. 2) 13
4.5 Agent definition in CrewAI (source: [1], Fig. 3) 13
4.6 Task definition in CrewAI (source: [1], Fig. 4) 14

5.1 Code generation and code check loop based on LangGraph+CrewAI (source: [1], Fig. 5) 15
5.2 Work-order processing with a crew of agents and tasks (source: [1], Fig. 6) . 16

B.1 Seminar slide: core concepts of LangGraph 24
B.2 Seminar slide: system architecture of the base paper 25
B.3 Seminar slide: workflow of the base paper 25
B.4 Seminar slide: key findings 26

C.1 Base paper: “Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI” (first page) 27

List of Tables

2.1 Summary of the literature survey 5

3.1 Development environment used in the base paper 8

5.1 Qualitative comparison of LLM application approaches 17

F.1 Mapping of the seminar topic to relevant UN Sustainable Development Goals (SDGs) 31

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CoT	Chain of Thought
FAISS	Facebook AI Similarity Search
HITL	Human-in-the-Loop
JS / TS	JavaScript / TypeScript
LLM	Large Language Model
MAS	Multi-Agent System
MIT	Massachusetts Institute of Technology (licence)
ML	Machine Learning
PM	Product Manager
RAG	Retrieval-Augmented Generation
SDG	Sustainable Development Goal
SOP	Standard Operating Procedure

Chapter 1

Introduction

Large language models have moved quickly from answering single questions to acting as the “brain” of software systems. A modern LLM can reason through a problem, write code, search the web and call other tools. Yet anyone who has tried to build something useful with one runs into the same problem: real tasks need many steps. A customer query may have to be classified, routed to the right department, answered with information from a database, and then checked before it is sent. A piece of generated code has to be run, tested and fixed, possibly several times.

These workflows are not straight lines. They **branch** (if the query is about billing, go one way; otherwise, another), they **loop** (retry until the answer is correct) and they need to **remember** earlier results. Two common ways of building LLM applications both struggle here. *Simple chains* run a fixed, one-way sequence of steps: they are predictable but cannot loop and have no shared state. *Free-running agents* let the LLM decide everything, including which tool to call next and when to stop: they are flexible but hard to control, test and debug, and a single wrong decision can derail the whole run.

LangGraph sits between these two extremes. It is an open-source library released by LangChain Inc. in 2024 under the MIT licence, available for Python and JavaScript/TypeScript [2]. Its design is inspired by graph-processing models such as Google’s Pregel [3] and Apache Beam. In LangGraph, the developer draws the application as a **graph**: each node is a step of work, each edge is a transition, and every node reads and updates one shared **state**. The developer decides which paths are allowed; the LLM does the work inside each step. The library works with any model provider, including OpenAI, Anthropic, Google and open-source models, and is used for single agents, multi-agent teams, retrieval-augmented generation (RAG) pipelines and workflows that need human approval.

The base paper for this seminar, by Duan and Wang [1], explores what happens when LangGraph is combined with CrewAI [4], a framework for organising role-based agents into a team. In their design, LangGraph owns the control flow (the order of steps, branching, loops and state), and CrewAI owns the teamwork (which agent does which task and how

agents collaborate). The authors demonstrate this combination on an email assistant, a code generation and review loop, and a ticket-processing system.

Objectives of the Seminar

- To understand why multi-step LLM applications need explicit orchestration, and where chains and free-running agents fall short.
- To explain LangGraph's core concepts: state, nodes, edges, conditional edges and checkpoints.
- To review related work on LLM agents and multi-agent systems.
- To study the LangGraph+CrewAI architecture, workflow and case studies presented in the base paper.
- To assess the advantages, limitations and applications of graph-based agent orchestration.

The rest of this report is organised as follows. Chapter 2 reviews related literature. Chapter 3 describes the tools and frameworks involved. Chapter 4 presents the methodology of the base paper, and Chapter 5 discusses its results. Chapters 6 to 8 cover advantages, disadvantages and applications, and Chapter 9 concludes the report.

Chapter 2

Literature Survey

This chapter summarises six studies that shaped the area of LLM agents and that help explain why a framework like LangGraph is needed. For each paper I note the method, what it showed, and the limitation that matters for building controllable multi-agent systems.

2.1 ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2023)

Yao et al. [5] proposed ReAct, in which the LLM alternates between three kinds of output: a *Thought* (reasoning about what to do), an *Action* (calling a tool such as a Wikipedia search) and an *Observation* (the tool's result). Interleaving reasoning with actions let the model gather facts as it went, and ReAct outperformed reasoning-only and acting-only baselines on HotpotQA, ALFWorld and WebShop. ReAct is now the basic pattern behind most tool-using agents. Its weakness is structural: it is a single linear loop with no persistent state, so a wrong step early in the run can cascade and derail the entire task.

2.2 Reflexion: Language Agents with Verbal Reinforcement Learning (Shinn et al., 2023)

Reflexion [6] lets an agent learn from its mistakes without any fine-tuning. After a failed attempt, the agent writes a short verbal reflection on what went wrong and stores it in memory, and the reflection is fed into the next attempt. This simple idea produced large gains on coding and decision-making benchmarks. However, Reflexion depends on a reliable success or failure signal, which many real tasks do not provide, and every retry costs additional LLM calls and time.

2.3 AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation (Wu et al., 2023)

AutoGen [7], an open-source framework from Microsoft, builds applications from several agents that solve a task by chatting with each other. Agents can combine LLMs, tools and human input, which makes the framework very flexible. The drawback is that the control flow emerges from the conversation itself. As a result, it is hard to predict what the system will do, and agents can loop or drift away from the task without strict control.

2.4 MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework (Hong et al., 2024)

MetaGPT [8] encodes Standard Operating Procedures (SOPs) from human software teams into prompts. Agents take on roles such as product manager, architect and engineer and work like an assembly line, passing structured documents to each other and cross-checking outputs to reduce errors. MetaGPT showed that structure reduces the inconsistency of chat-based teams. However, its pipeline is fixed and designed mainly for software projects, and using many agents leads to a high token cost.

2.5 A Survey on Large Language Model based Autonomous Agents (Wang et al., 2023)

Wang et al. [9] reviewed the fast-growing literature on LLM-based autonomous agents and proposed a unified framework with four modules: *profile* (the agent's role), *memory*, *planning* and *action*. The survey also covers applications and evaluation methods. It is a valuable map of the field, but as a survey it offers no implementation or orchestration tool, and the field is moving faster than surveys can track.

2.6 Agent AI with LangGraph: A Modular Framework for Enhancing Machine Translation Using LLMs (Wang and Duan, 2024)

In earlier work by the same authors as the base paper [10], LangGraph analyses the input text and routes it to a specialised translation agent, with separate agents for English, French and Japanese running on models such as GPT-4o. The paper shows LangGraph's routing

in a concrete application. Its evaluation, however, covers only a few languages and is small in scale, and the system relies on a paid proprietary model.

Summary

Taken together, these studies show steady progress: agents that act (ReAct), agents that learn from failure (Reflexion), agents that converse (AutoGen) and agents that follow procedures (MetaGPT). They also show a recurring gap. Either the flow is rigid and cannot loop or branch, or it is left to the model and becomes hard to control. What is missing is a way for the developer to define the allowed paths explicitly, with shared state, loops and recovery, while still letting the LLM do the intelligent work inside each step. This is the gap that LangGraph, and the LangGraph+CrewAI combination in the base paper, sets out to fill.

Table 2.1: Summary of the literature survey

Paper	**Key idea**	**Limitation**
ReAct [5]	Interleaves reasoning steps with tool actions	One linear loop; no saved state; errors cascade
Reflexion [6]	Agent reflects on failures and retries	Needs good feedback; extra LLM calls
AutoGen [7]	Agents solve tasks by conversing	Conversation-driven flow is hard to control
MetaGPT [8]	Role-based agents follow SOPs	Rigid pipeline built for software tasks
Agent Survey [9]	Unified framework: profile, memory, planning, action	No concrete orchestration tool
LangGraph MT [10]	LangGraph routes text to translation agents	Few languages; small evaluation

Chapter 3

Materials and Methods

Unlike a model-training study, the base paper does not use a dataset. Its “materials” are the frameworks and tools from which the multi-agent system is built. This chapter describes them and the criteria the paper uses to judge the result.

3.1 LangGraph: Core Concepts

LangGraph is a framework for building stateful, multi-step LLM applications as graphs [2]. Compared with other LLM frameworks, it offers three things the base paper highlights: **loops**, **controllability** and **persistent memory** [1]. It supports many kinds of control flow (single-agent, multi-agent, hierarchical and sequential) and is meant to handle complex, real-world scenarios. The building blocks are listed below and shown in Fig. 3.1.

- **State:** shared data, usually a typed dictionary, that every node reads and updates. Reducer functions define how an update is merged, for example appending to a list of messages instead of overwriting it.
- **Node:** an ordinary Python function that performs one step of work, such as calling the LLM or a tool, and returns the parts of the state it changes.
- **Edge:** a fixed transition: after node A, run node B.
- **Conditional edge:** a router function that looks at the current state and picks the next node. This is how branching and loops are expressed.
- **START and END:** special entry and exit points of the graph.
- **Checkpointer:** saves the state after every step, so a run can be paused, inspected, resumed after a crash, or continued after a human approves it.

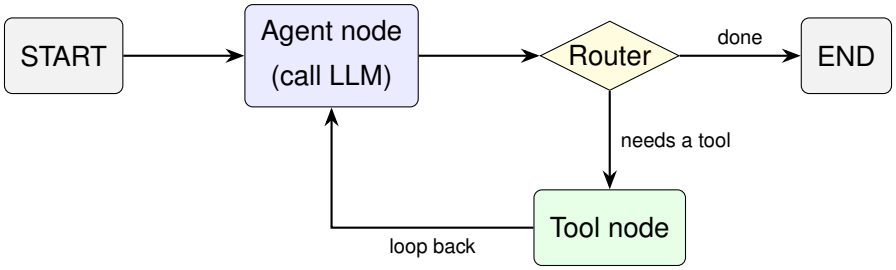


Figure 3.1: A minimal LangGraph agent: nodes, a conditional edge (router) and a loop

Building a LangGraph application takes five steps: (1) define the state, (2) add nodes as functions, (3) connect them with edges, (4) compile the graph and (5) invoke it. The following minimal example answers a question with one LLM call:

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):          # 1. define the state
    question: str
    answer: str

def answer(state: State):        # 2. a node is a plain function
    return {"answer": llm.invoke(state["question"])}

g = StateGraph(State)
g.add_node("answer", answer)
g.add_edge(START, "answer")      # 3. connect edges
g.add_edge("answer", END)
app = g.compile()                # 4. compile
app.invoke({"question": "What is LangGraph?"})   # 5. run
```

3.2 CrewAI Framework

CrewAI [4] is an open-source framework for coordinating role-playing, autonomous AI agents so that they can cooperate on complex problems. Developers define each agent with a specific role, objective and set of tools. Its four building blocks are:

- **Agent:** an autonomous unit that can perform tasks, make decisions and communicate with other agents. It is defined by a role, a goal and a backstory.
- **Task:** a specific piece of work carried out by an agent, including its description, the agent that executes it and the tools it needs.
- **Tool:** a capability an agent can use, such as reading email or searching documents.
- **Crew:** a team of agents together with the process that orchestrates their tasks.

CrewAI supports APIs such as OpenAI and Ollama, and its key features include role-customised agents, automatic task delegation and flexible task management [1]. It is aimed at multi-step workflows, decision-making and problem-solving.

3.3 Development Environment

The base paper builds its application cases with the following components [1]:

Table 3.1: Development environment used in the base paper

Component	**Tool**	**Purpose**
Multi-agent framework	CrewAI	Organising and coordinating collaborative work among AI agents
Graph workflow	LangGraph	Managing state sharing and process control between graph nodes
Workflow tracking	LangSmith [11]	Monitoring and auditing the execution of workflows
Embedding model	Ollama embeddings	Converting work-order text into vectors for retrieval
Vector database	FAISS [12]	Storing and retrieving vectors, which speeds up data access

3.4 Evaluation Criteria

The base paper is an exploratory case study. It does not report accuracy, latency or cost figures. Instead, it judges the LangGraph+CrewAI combination qualitatively, on four criteria that also guide this report:

- **Controllability:** whether the developer can define and restrict the flow of the application precisely.
- **Information flow:** how efficiently state and results pass between agents and steps.
- **Collaboration:** how well work is divided among agents with different roles.
- **Extensibility:** how easily existing tools and agents, for example those built with LangChain, can be integrated.

Chapter 4

Methodology

4.1 Design Rationale

The base paper starts from a simple observation: complex tasks are too much for a single LLM agent, and multi-agent systems need two things to work well, namely precise control of the flow and good teamwork [1]. Rather than making one framework do both, the authors give each job to the framework best suited to it. LangGraph's graph architecture provides precise control and efficient information flow between agents. CrewAI's role-based crews provide intelligent task allocation and resource management. The paper studies two questions: (1) how to design the architecture of agents on LangGraph for more precise control, and (2) how to enhance agents with CrewAI so that they can complete different tasks.

4.2 System Architecture

The architecture has two layers that meet at the shared state (Fig. 4.1).

- **LangGraph, the control layer.** The user's task enters the graph. Nodes represent agent steps, and edges with conditional routing decide which step runs next. The shared state carries data between steps.
- **CrewAI, the team layer.** Inside a node, a crew of agents does the actual work. Each agent has a role and a goal, tasks are assigned to agents, and the agents collaborate to produce the node's result.

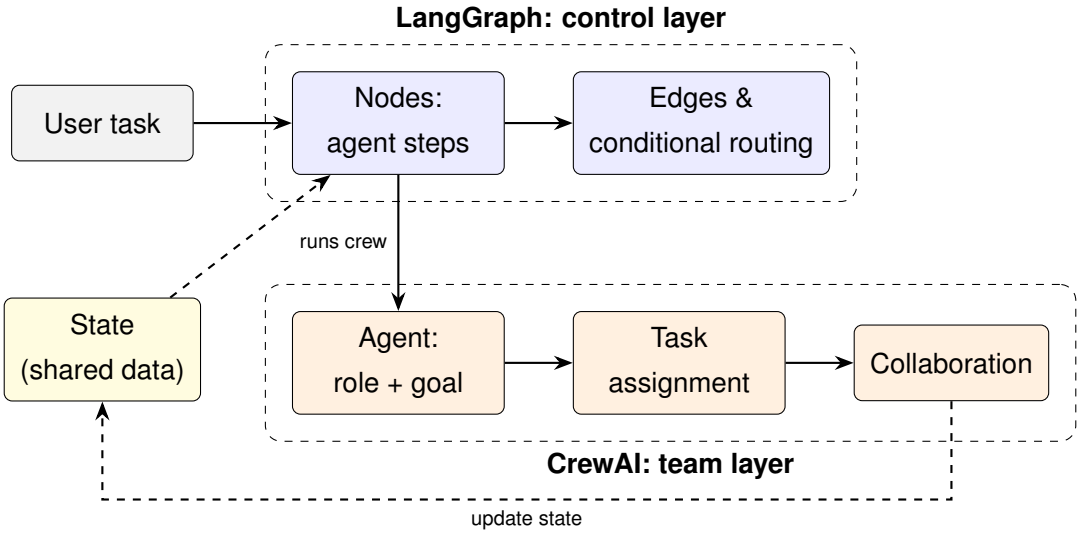


Figure 4.1: Two-layer architecture: LangGraph controls the flow, CrewAI performs the work inside each node

4.3 Integrating LangGraph with CrewAI

According to the authors, the integrated framework is a powerful tool for complex task management. It optimises task execution through flexible workflows, collaboration between agents and graph-structured management. It also supports custom development by integrating with existing tools. CrewAI keeps the process efficient through clear task allocation and role definitions, and because it integrates with LangChain, developers who already use LangChain can bring in their existing agents with little effort [1].

In practice, the division of labour is straightforward:

- **LangGraph owns the control flow:** the order of steps, branching, loops and state.
- **CrewAI owns the teamwork:** which role-based agent does which task, and how the agents collaborate.

4.4 Workflow

Fig. 4.2 shows how a task moves through the combined system. LangGraph receives the task and routes it to the next node. The CrewAI crew in that node performs the task, and the result is written back to the shared state. A conditional edge then checks whether the work is complete. If it is not, the flow loops to the next step; if it is, the graph ends.

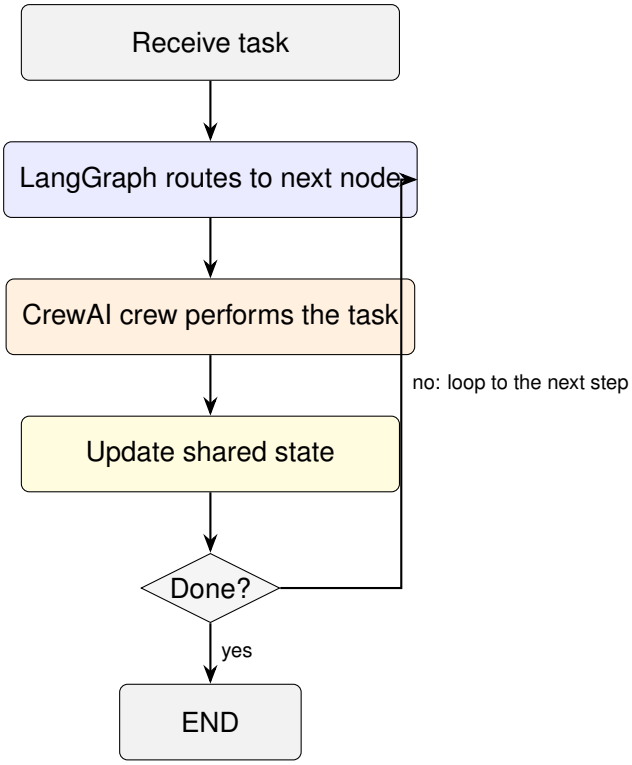


Figure 4.2: Workflow of the LangGraph+CrewAI system

4.5 Worked Example: Automatic Email Handling

To make the method concrete, the base paper takes the automatic email composition and sending example from the CrewAI website and rebuilds it on LangGraph+CrewAI [1]. The task is broken into manageable steps, and their execution is automated.

Fig. 4.3 shows the graph. The flow starts at `check_new_emails`. A conditional edge either continues to `draft_responses`, when there are new emails, or goes straight to `wait_next_run`. After waiting, the graph loops back to check again. The loop, which a simple chain cannot express, is what turns a one-off script into a continuously running assistant.

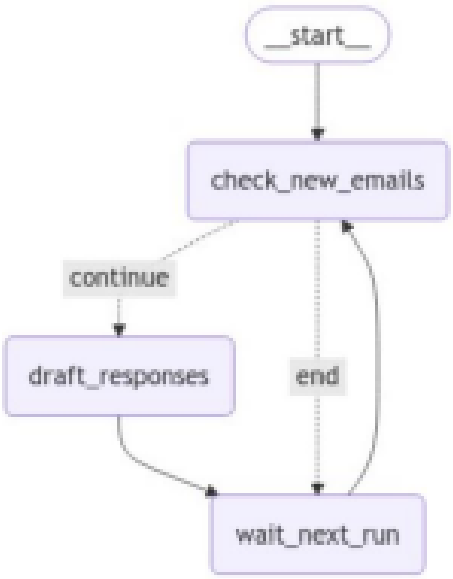


Figure 4.3: Email workflow graph built with LangGraph+CrewAI (source: Duan and Wang [1], Fig. 1)

The code that builds this graph (Fig. 4.4) follows the five steps described in Chapter 3. The interesting line is the node `draft_responses`, whose function is `EmailFilterCrew().kickoff`. Here an entire CrewAI crew runs as a single LangGraph node:

```
workflow = StateGraph(EmailsState)
workflow.add_node("check_new_emails", nodes.check_email)
workflow.add_node("wait_next_run", nodes.wait_next_run)
workflow.add_node("draft_responses", EmailFilterCrew().kickoff)
workflow.set_entry_point("check_new_emails")
workflow.add_conditional_edges(
    "check_new_emails", nodes.new_emails,
    {"continue": "draft_responses", "end": "wait_next_run"})
workflow.add_edge("draft_responses", "wait_next_run")
workflow.add_edge("wait_next_run", "check_new_emails")
self.app = workflow.compile()
```

```python
from dotenv import load_dotenv
load_dotenv()
from langgraph.graph import StateGraph
from .state import EmailsState
from .nodes import Nodes
from .crew.crew import EmailFilterCrew

class Workflow():
    def __init__(self):
        nodes = Nodes()
        workflow = StateGraph(EmailsState)
        workflow.add_node("check_new_emails", nodes.check_email)
        workflow.add_node("wait_next_run", nodes.wait_next_run)
        workflow.add_node("draft_responses", EmailFilterCrew().kickoff)
        workflow.set_entry_point("check_new_emails")
        workflow.add_conditional_edges(
                "check_new_emails",
                nodes.new_emails,
                {
                    "continue": 'draft_responses',
                    "end": "wait_next_run"
                }
        )
        workflow.add_edge('draft_responses', 'wait_next_run')
        workflow.add_edge('wait_next_run', 'check_new_emails')
        self.app = workflow.compile()
```

Figure 4.4: LangGraph+CrewAI workflow code (source: [1], Fig. 2)

Inside the crew, each agent is defined by its role, goal and backstory (Fig. 4.5). For example, the “Senior Email Analyst” filters out newsletters and promotional mail. A task (Fig. 4.6) then describes exactly what the agent must do, which output format is expected, and which agent performs it.

```python
class EmailFilterAgents():
    def __init__(self):
        self.gmail = GmailToolkit()

    def email_filter_agent(self):
        return Agent(
            role='Senior Email Analyst',
            goal="Filter out non-essential emails like newsletters and promotional content",
            backstory=dedent("""\
                As a Senior Email Analyst, you have extensive experience in email content analysis.
                You are adept at distinguishing important emails from spam, newsletters, and other
                irrelevant content. Your expertise lies in identifying key patterns and markers that
                signify the importance of an email."""),
            verbose=True,
            allow_delegation=False
        )
```

Figure 4.5: Agent definition in CrewAI (source: [1], Fig. 3)

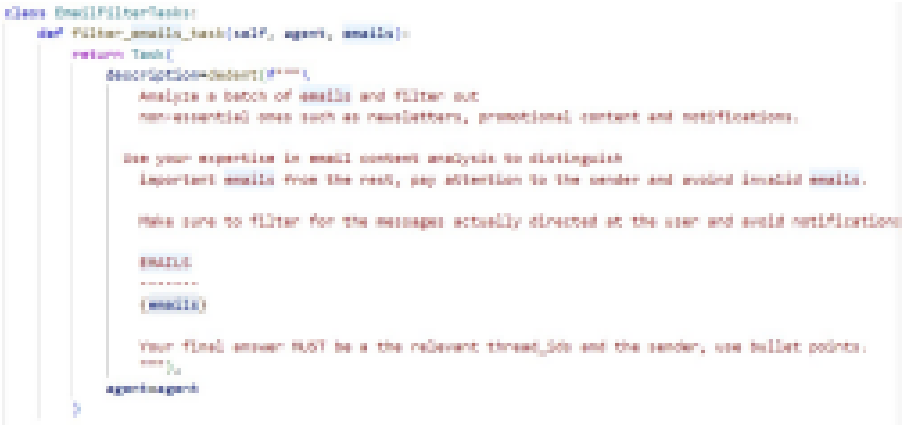


Figure 4.6: Task definition in CrewAI (source: [1], Fig. 4)

Chapter 5

Results and Performance

The base paper reports its results as application case studies. It shows that the LangGraph+CrewAI design can be used to build working multi-agent applications and describes what the combination improves. This chapter presents those cases, the key findings and a discussion of what the results do and do not show.

5.1 Application Case 1: Code Generation with Code Review

Using LangGraph+CrewAI, the authors built a multi-agent application that combines code generation with code review (Fig. 5.1). A question goes to a *code generation* node, which produces code. A *code check* node then tests it. If the check fails, the flow loops back to generation with the feedback; if it passes, the code is returned as the solution. Because status data and feedback are shared in real time through the graph state, the authors report that the efficiency of code generation improved [1]. This is the same write, test and fix loop that the Reflexion and ReAct papers motivate, but here the loop is an explicit, inspectable edge in the graph rather than something left to the model.

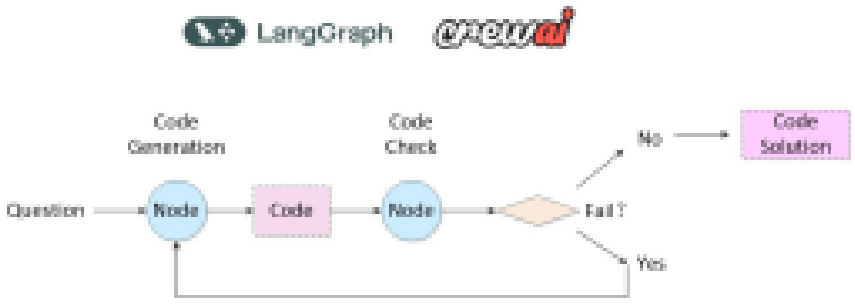


Figure 5.1: Code generation and code check loop based on LangGraph+CrewAI (source: [1], Fig. 5)

5.2 Application Case 2: Automatic Work-Order Processing

The second case automates the auditing and forwarding of work orders, or tickets (Fig. 5.2). The text of each ticket is converted into vectors with an Ollama embedding model and stored in FAISS, so that similar past tickets can be retrieved. Several CrewAI agents, each with its own task, then analyse the ticket in depth. By sharing their understanding of the content, the agents can interpret tickets more accurately, and the authors report that this improved the efficiency of ticket processing [1].

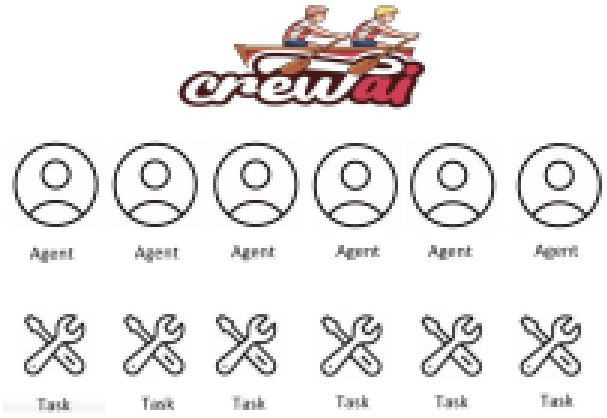


Figure 5.2: Work-order processing with a crew of agents and tasks (source: [1], Fig. 6)

5.3 Key Findings

From these cases, the authors draw four findings:

- **Efficient information flow:** LangGraph's graph architecture improves how efficiently information passes between agents.
- **Precise control:** designing agents on LangGraph gives precise control over the multi-agent workflow.
- **Better collaboration:** CrewAI's intelligent task allocation and resource management improve teamwork and system performance.
- **Complex tasks:** agents divide the work and collaborate on tasks that a single agent struggles with.

5.4 Comparison with Other Approaches

Table 5.1 compares the approaches discussed in this report on the properties that matter for building controllable multi-agent applications. The comparison is qualitative and based on the literature reviewed in Chapter 2.

Table 5.1: Qualitative comparison of LLM application approaches

Approach	**Flow defined by**	**Loops**	**Persistent state**	**Human-in-the-loop**	**Role-based teamwork**	**Predictability**
Simple chain	Developer	No	No	No	No	High
ReAct agent	LLM	Yes	No	Limited	No	Low
AutoGen	Conversation	Yes	Partial	Yes	Yes	Low
MetaGPT	Fixed SOP	Limited	Documents	No	Yes	Medium
LangGraph alone	Developer (graph)	Yes	Yes (checkpoints)	Yes	Manual	High
LangGraph + CrewAI	**Developer (graph)**	**Yes**	**Yes**	**Yes**	**Yes (crews)**	**High**

5.5 Discussion

The base paper succeeds in showing *how* LangGraph and CrewAI fit together, and its three cases make the division of responsibilities easy to understand: the graph decides what happens next, and the crew decides who does it. The examples are also practical. Email triage, code review and ticket handling are everyday tasks in many organisations.

At the same time, the results should be read for what they are. The paper reports no quantitative measurements: no accuracy or success rates, no latency or cost figures, and no comparison against a single-agent baseline or another framework. Claims such as “improved efficiency” are therefore based on the authors’ observation of the cases, not on measured experiments. A natural next step would be to evaluate the combined framework on a benchmark, for example measuring the task success rate, the number of LLM calls and the cost against a single agent and against LangGraph alone. Even so, the paper is a useful and approachable guide to designing multi-agent applications in which control and collaboration are handled by separate, well-defined layers.

Chapter 6

Advantages

- **Explicit control:** the developer decides which paths the application may take. The LLM fills in each step but cannot invent a new flow, which makes behaviour easier to predict and to audit.
- **Shared state:** every step sees the same up-to-date data, so information does not have to be copied between agents or squeezed into prompts.
- **Loops and retries:** a step can be repeated until a condition is met, for example regenerating code until its tests pass. Plain chains cannot do this.
- **Persistence:** checkpointers save the state after every step, so an application can pause, resume and recover after a failure without starting over.
- **Human-in-the-loop:** the graph can stop at any node and wait for a person to review or edit the state before continuing, which is important for sensitive actions such as sending emails.
- **Easy to debug:** each node is a plain function that can be tested and traced on its own. Tools such as LangSmith show exactly which path a run took.
- **Model-agnostic:** LangGraph and CrewAI both work with many LLM providers, including OpenAI, Anthropic, Google and local models through Ollama.
- **Clear teamwork with CrewAI:** roles, goals and tasks give each agent a well-defined job, and existing LangChain agents can be brought into a crew with little effort.

Chapter 7

Disadvantages

- **Learning curve:** graphs, typed state, reducers and checkpointers take time to learn, and combining two frameworks adds a second set of concepts.
- **More code for simple tasks:** a task that a basic chain handles in a few lines needs more setup as a graph.
- **Large graphs get complex:** with many nodes and conditional routes, the flow becomes hard to visualise and reason about.
- **Cost and latency:** loops and multiple agents mean more LLM calls, which increases both the running cost and the response time.
- **Still depends on the LLM:** the graph constrains the paths, but a wrong model output can still pick a wrong route or produce a poor result inside a node.
- **Fast-changing ecosystem:** the APIs of both frameworks change between versions, so code and tutorials go out of date quickly.
- **No quantitative evaluation in the base paper:** the benefits are shown through case studies, so they are not yet backed by measured results.

Chapter 8

Applications

- **Customer support:** a router node classifies each query and sends it to a billing agent, a technical agent or a human, with the conversation kept in the shared state.
- **RAG question answering:** the graph retrieves documents, grades their relevance and, if they are poor, rewrites the query and searches again before answering.
- **Code generation and bug fixing:** write, test and fix in a loop, as in the base paper's code review case and in the authors' later study on automated bug fixing with LangGraph [13].
- **Machine translation:** the input text is routed to language-specific translation agents [10].
- **Data analysis:** modular agents build and run machine learning pipelines on Apache Spark [14].
- **Multi-agent teams:** crews for research, writing and review, in which each agent plays a role, as proposed in the base paper [1].
- **Ticket and workflow automation:** auditing, classifying and forwarding work orders, with retrieval of similar past cases from a vector database.
- **AI software engineering:** graph-based harnesses that design, generate, test and repair software automatically, which is the direction of my major project, VibeStack, built on LangGraph.

Chapter 9

Conclusion

LangGraph turns an LLM application into a clear graph of steps with shared state. It adds what simple chains lack, namely branching, loops, memory and human control, while avoiding the unpredictability of fully autonomous agents. The developer decides which paths are allowed, and the model does the intelligent work inside each step.

The base paper by Duan and Wang shows how far this idea can go when it is combined with CrewAI. LangGraph controls the flow and the state, CrewAI organises role-based agents into teams, and together they handle tasks that a single agent struggles with. The email assistant, the code generation and review loop and the ticket-processing system make the approach concrete and show that it applies to everyday problems.

The paper's main weakness is its evaluation. The benefits are shown through case studies rather than measured experiments, so the efficiency gains it describes still need to be confirmed with benchmarks covering success rate, cost and latency. Future work could also study how such systems behave when the number of agents grows, and how to keep large graphs understandable.

Overall, graph-based orchestration is becoming a standard way to build reliable agentic AI. Studying it for this seminar has also been directly useful to me: the same ideas of explicit state, conditional routing, loops that repair their own output, and checkpoints are the foundation of my major project.

Appendix A

Bibliography

[1] Z. Duan and J. Wang, “Exploration of LLM multi-agent application implementation based on LangGraph+CrewAI,” *arXiv preprint arXiv:2411.18241*, 2024.

[2] LangChain Inc., “LangGraph documentation,” https://langchain-ai.github.io/langgraph/, 2025.

[3] G. Malewicz, M. H. Austern, A. J. C. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski, “Pregel: A system for large-scale graph processing,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2010, pp. 135–146.

[4] CrewAI Inc., “CrewAI documentation,” https://docs.crewai.com, 2025.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629.

[6] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2303.11366.

[7] Q. Wu, G. Bansal, J. Zhang, Y. Wu *et al.*, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.

[8] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng *et al.*, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2308.00352.

[9] L. Wang, C. Ma, X. Feng, Z. Zhang *et al.*, “A survey on large language model based autonomous agents,” *arXiv preprint arXiv:2308.11432*, 2023.

[10] J. Wang and Z. Duan, “Agent AI with LangGraph: A modular framework for enhancing machine translation using large language models,” *arXiv preprint arXiv:2412.03801*, 2024.

[11] LangChain Inc., “LangSmith documentation,” https://docs.smith.langchain.com, 2025.

[12] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[13] J. Wang and Z. Duan, “Empirical research on utilizing LLM-based agents for automated bug fixing via LangGraph,” *arXiv preprint arXiv:2502.18465*, 2025.

[14] ——, “Intelligent spark agents: A modular LangGraph framework for scalable, visualized, and enhanced big data machine learning workflows,” *arXiv preprint arXiv:2412.01490*, 2024.

Appendix B

Presentation Slides

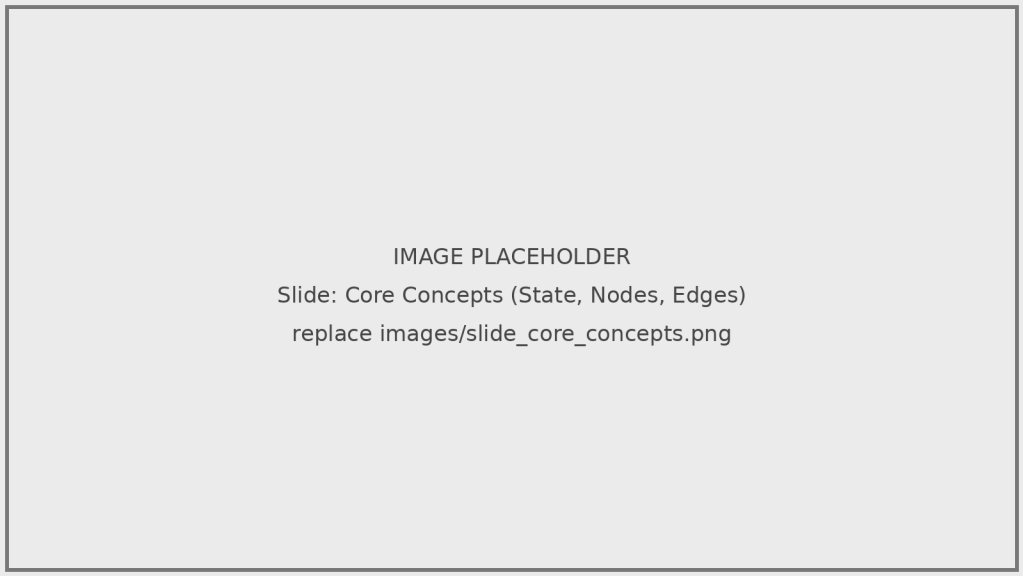


Figure B.1: Seminar slide: core concepts of LangGraph

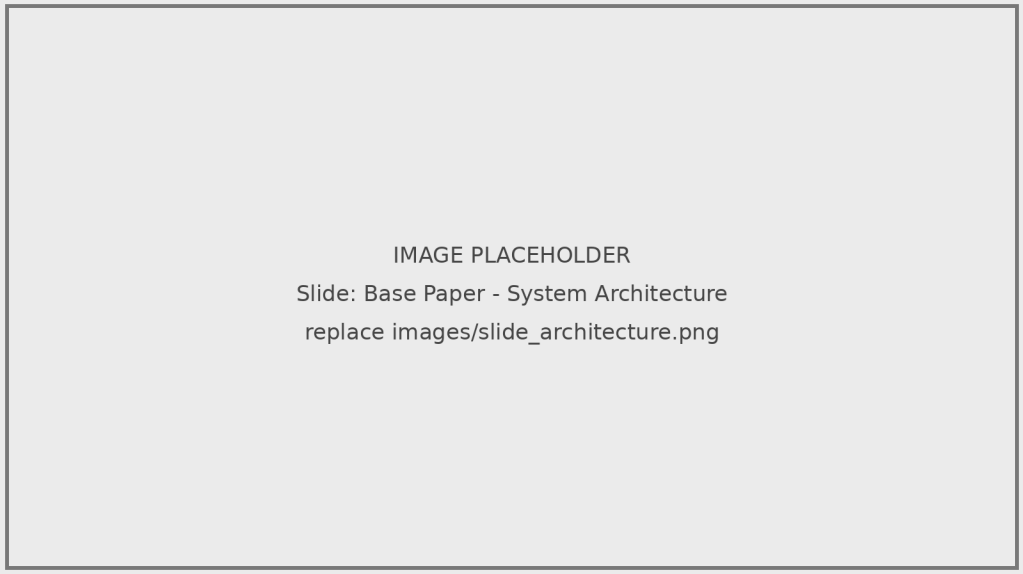


Figure B.2: Seminar slide: system architecture of the base paper

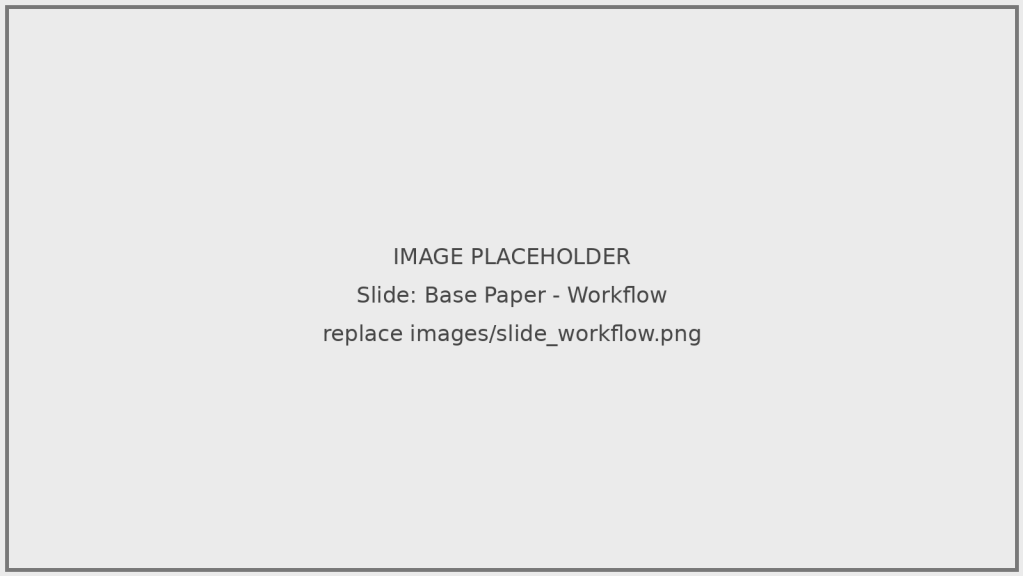


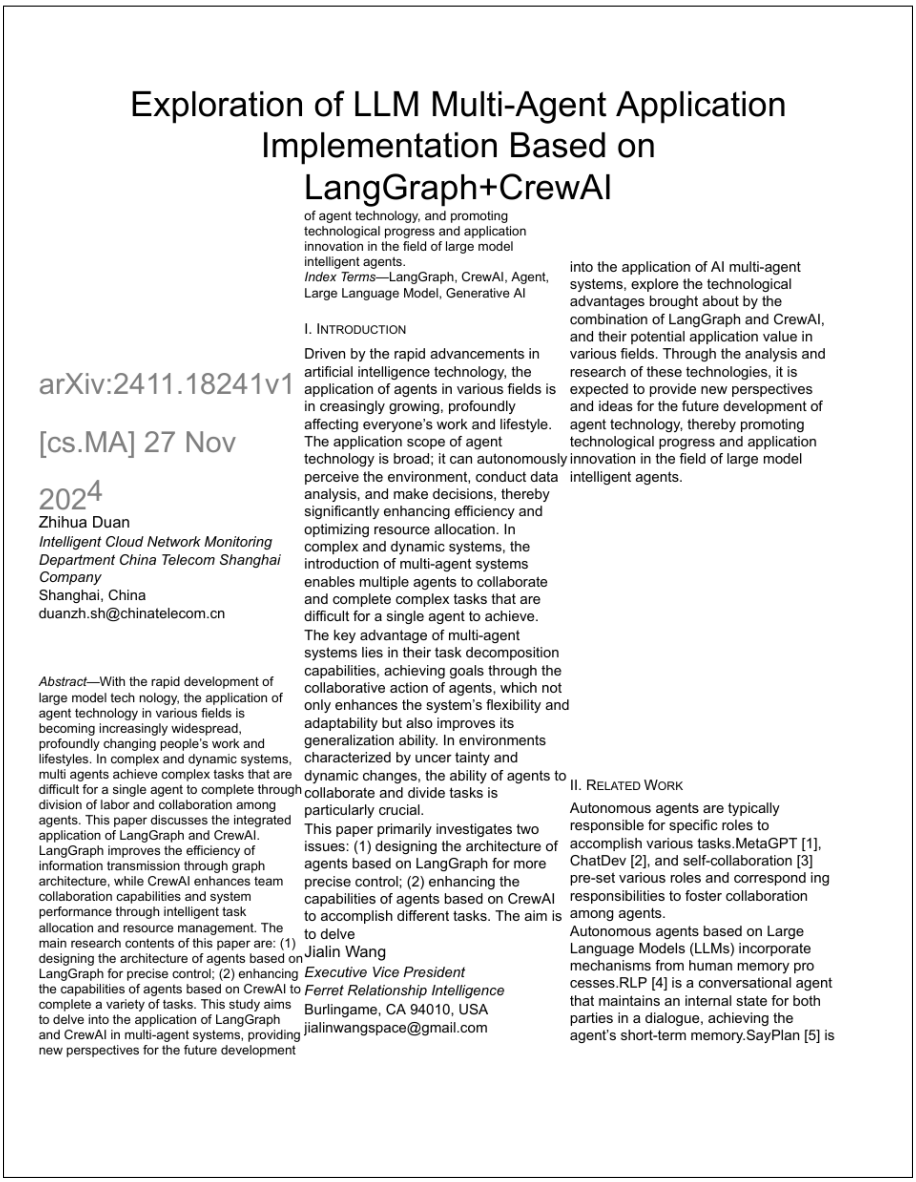
Figure B.3: Seminar slide: workflow of the base paper



Figure B.4: Seminar slide: key findings

Appendix C

Sample Reference Papers



Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI

arXiv:2411.18241v1 [cs.MA] 27 Nov 2024

Zhihua Duan
Intelligent Cloud Network Monitoring Department China Telecom Shanghai Company
Shanghai, China
duanzh.sh@chinatelecom.cn

Abstract—With the rapid development of large model tech nology, the application of agent technology in various fields is becoming increasingly widespread, profoundly changing people's work and lifestyles. In complex and dynamic systems, multi agents achieve complex tasks that are difficult for a single agent to complete through division of labor and collaboration among agents. This paper discusses the integrated application of LangGraph and CrewAI. LangGraph improves the efficiency of information transmission through graph architecture, while CrewAI enhances team collaboration capabilities and system performance through intelligent task allocation and resource management. The main research contents of this paper are: (1) designing the architecture of agents based on LangGraph for precise control; (2) enhancing the capabilities of agents based on CrewAI to complete a variety of tasks. This study aims to delve into the application of LangGraph and CrewAI in multi-agent systems, providing new perspectives for the future development of agent technology, and promoting technological progress and application innovation in the field of large model intelligent agents.

Index Terms—LangGraph, CrewAI, Agent, Large Language Model, Generative AI

I. Introduction

Driven by the rapid advancements in artificial intelligence technology, the application of agents in various fields is in creasingly growing, profoundly affecting everyone's work and lifestyle. The application scope of agent technology is broad; it can autonomously perceive the environment, conduct data analysis, and make decisions, thereby significantly enhancing efficiency and optimizing resource allocation. In complex and dynamic systems, the introduction of multi-agent systems enables multiple agents to collaborate and complete complex tasks that are difficult for a single agent to achieve. The key advantage of multi-agent systems lies in their task decomposition capabilities, achieving goals through the collaborative action of agents, which not only enhances the system's flexibility and adaptability but also improves its generalization ability. In environments characterized by uncer tainty and dynamic changes, the ability of agents to collaborate and divide tasks is particularly crucial.

This paper primarily investigates two issues: (1) designing the architecture of agents based on LangGraph for more precise control; (2) enhancing the capabilities of agents based on CrewAI to accomplish different tasks. The aim is to delve into the application of AI multi-agent systems, explore the technological advantages brought about by the combination of LangGraph and CrewAI, and their potential application value in various fields. Through the analysis and research of these technologies, it is expected to provide new perspectives and ideas for the future development of agent technology, thereby promoting technological progress and application innovation in the field of large model intelligent agents.

Jialin Wang
Executive Vice President
Ferret Relationship Intelligence
Burlingame, CA 94010, USA
jialinwangspace@gmail.com

II. Related Work

Autonomous agents are typically responsible for specific roles to accomplish various tasks.MetaGPT [1], ChatDev [2], and self-collaboration [3] pre-set various roles and correspond ing responsibilities to foster collaboration among agents.

Autonomous agents based on Large Language Models (LLMs) incorporate mechanisms from human memory pro cesses.RLP [4] is a conversational agent that maintains an internal state for both parties in a dialogue, achieving the agent's short-term memory.SayPlan [5] is

Figure C.1: Base paper: "Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI" (first page)

Appendix D

Viva Questions and Answers

Common Viva Questions

1. **What is LangGraph?**
   Answer: LangGraph is an open-source library from LangChain Inc. for building stateful, multi-step LLM applications as a graph of nodes (steps) and edges (transitions) that share one state object.

2. **How is LangGraph different from a LangChain chain?**
   Answer: A chain runs a fixed, one-way sequence of steps. A LangGraph graph can branch with conditional edges, loop back to earlier steps, keep shared state and save checkpoints, so it can retry, pause and resume.

3. **What are state, nodes and edges?**
   Answer: The state is shared data, usually a typed dictionary, that every node reads and updates. A node is a function that performs one step. An edge is a transition between nodes, and a conditional edge uses a router function to choose the next node from the current state.

4. **What is a checkpointer and why is it useful?**
   Answer: A checkpointer saves the graph state after every step. This allows a run to resume after a crash, to be inspected for debugging, and to pause for human approval before continuing.

5. **What does CrewAI add to LangGraph in the base paper?**
   Answer: CrewAI provides role-based teamwork. Agents are defined with roles, goals and tools, and tasks are allocated among them. LangGraph controls the order of steps and the state, while a CrewAI crew performs the work inside a node.

6. **Explain the email example from the base paper.**
   Answer: The graph starts at `check_new_emails`. If there are new emails, it continues

to `draft_responses`, which runs a CrewAI crew; otherwise it goes to `wait_next_run`. After waiting, it loops back to check again.

7. **What tools did the authors use besides LangGraph and CrewAI?**
   Answer: LangSmith for tracking and auditing workflows, an Ollama embedding model to turn work-order text into vectors, and FAISS as the vector database for fast retrieval.

8. **How does LangGraph compare with AutoGen?**
   Answer: In AutoGen, the flow emerges from the conversation between agents, which is flexible but hard to predict. In LangGraph, the developer defines the allowed paths explicitly, which makes behaviour more controllable and easier to debug.

9. **What are the limitations of the base paper?**
   Answer: The results are qualitative case studies. The paper gives no measurements of accuracy, cost or latency and no comparison with a baseline, so the reported efficiency gains are not yet backed by experiments.

10. **Where can LangGraph be applied?**
    Answer: In customer support routing, RAG question answering, code generation and bug fixing, machine translation, data analysis pipelines, multi-agent research and writing teams, and ticket automation.

Appendix E

Similarity Index Report

[Insert Similarity Index Report Here]

Appendix F

Alignment with Sustainable Development Goals (SDG)

Table F.1: Mapping of the seminar topic to relevant UN Sustainable Development Goals (SDGs)

SDG	**Rationale for Alignment**
SDG 4: Quality Education	Graph-based agents can power tutoring and question-answering assistants, and the clear structure of LangGraph makes agentic AI easier for students to learn.
SDG 8: Decent Work and Economic Growth	Automating routine work such as email triage and ticket processing frees people for more meaningful tasks and raises productivity.
SDG 9: Industry, Innovation and Infrastructure	Open-source orchestration frameworks lower the barrier to building reliable AI systems, supporting innovation in industry and research.
SDG 17: Partnerships for the Goals	Open-source, model-agnostic tools encourage collaboration among researchers, developers and organisations.